

SYNTH-04: Private Synthetic Locations

Series: SYNTH — Synthetic Monitoring | **Notebook:** 4 of 6 | **Created:** December 2025 | **Last Updated:** 07/08/2026

Monitoring Internal Applications from Your Infrastructure

This notebook covers deploying and managing private synthetic locations (ActiveGates) for monitoring internal applications, APIs, and services not accessible from the public internet.

Table of Contents

1. [Why Private Locations?](#)
 2. [Architecture](#)
 3. [Synthetic-Enabled ActiveGate](#)
 4. [Deployment Options](#)
 5. [Configuration](#)
 6. [Monitoring Private Location Health](#)
 7. [Troubleshooting](#)
-

Prerequisites

-  Access to a Dynatrace environment with Synthetic Monitoring

-  Completed SYNTH-01 through SYNTH-03
-  Infrastructure access to deploy ActiveGate (for setup)

1. Why Private Locations?

Public vs Private Locations

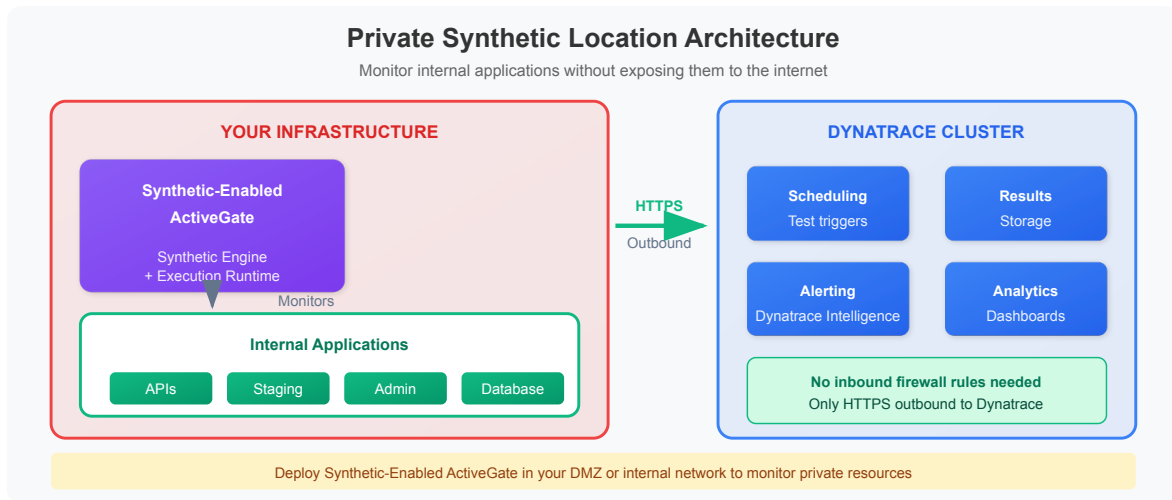
Aspect	Public Locations	Private Locations
Hosting	Dynatrace cloud	Your infrastructure
Access	Public internet only	Internal networks
Maintenance	Managed by Dynatrace	Managed by you
Security	External perspective	Behind firewall
Latency	Varies by region	Local network

Use Cases for Private Locations

Scenario	Description
Internal APIs	Services not exposed to internet
VPN-only apps	Applications requiring VPN access
Pre-production	Staging/dev environments
Security compliance	Data must not leave network
Low latency testing	Local performance baselines
Isolated networks	Air-gapped or restricted networks

2. Architecture

Private synthetic locations use ActiveGates deployed within your infrastructure to execute monitors against internal applications:



Key Points

- **Outbound only:** ActiveGate initiates all connections
- **No inbound ports:** No firewall changes for external access
- **Local execution:** Tests run inside your network
- **Results upload:** Only metrics/results sent to Dynatrace

3. Synthetic-Enabled ActiveGate

ActiveGate Capabilities

ActiveGate can serve multiple purposes:

Capability	Description
Synthetic	Private synthetic location
Metrics	Metric ingestion endpoint
Logs	Log ingestion endpoint
API	Cluster API proxy
Extensions	Extension execution

Synthetic Engine Requirements

Resource	Minimum	Recommended
CPU	2 cores	4+ cores
RAM	4 GB	8+ GB
Disk	20 GB	50+ GB
Network	HTTPS outbound	Low latency to targets

Browser Monitor Requirements

For browser monitors, additional requirements:

- Chrome/Chromium browser
- Display server (X11 or headless)
- Additional RAM for browser instances

Breaking change (SaaS 1.343, July 2026 — staged tenant rollout from mid-July): private Synthetic locations require **ActiveGate 1.331 or newer** once 1.343 reaches your tenant, as part of migrating private-location scoping from **management zones to security context**. Upgrade any AG below 1.331 before the enforcement reaches your tenant, and review location access controls after the migration — the security-context model replaces MZ-based scoping (consistent with the platform-wide management-zone retirement; the MZ2POL series covers the broader migration). SaaS 1.343 also adds IAM-based access control for Synthetic Monitoring and API support for assigning security context to synthetic monitors.

4. Deployment Options

Option 1: Linux/Windows Installer

```
# Download from Dynatrace Hub
# Settings → Deployment status → ActiveGate

# Linux installation
sudo /bin/sh Dynatrace-ActiveGate-Linux-x86-*.sh \
  --enable-synthetic

# Verify synthetic capability
sudo systemctl status dynatracegateway
```

Option 2: Container Deployment

```
# docker-compose.yml
version: '3'
services:
  activegate:
    image: dynatrace/dynatrace-activegate:latest
    environment:
      - DT_TENANT=abc12345
      - DT_CAPABILITIES=synthetic
      - DT_API_TOKEN=${DT_API_TOKEN}
    volumes:
      - activegate-data:/var/lib/dynatrace/gateway
    ports:
      - "9999:9999"
```

Option 3: Kubernetes / OpenShift

```
# Use Dynatrace Operator with ActiveGate CRD
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
spec:
  activeGate:
    capabilities:
      - synthetic-monitoring
    resources:
      requests:
        cpu: "500m"
        memory: "512Mi"
```

⚠ Breaking change for "Latest Dynatrace" tenants (Sprint 1.339):

Kubernetes / OpenShift private synthetic-location pod templates now require the environment variable `METRIC_3RD_GEN_ENABLED` on the synthetic-location pod. Without it, deployments on Latest tenants fail or run without 3rd-generation metric emission. Re-pull the template from **Settings → Synthetic → Private locations** or add this env var to existing manifests before/concurrent with the Latest Dynatrace transition, and schedule it through change management.

```
// List all synthetic locations
// Note: 'type' (PRIVATE/PUBLIC), 'status', 'city', 'countryCode' fields are
// not available
// Private locations are typically identified by their naming convention
fetch dt.entity.synthetic_location
| fields id, entity.name
| sort entity.name asc
| limit 50
```

```
// Count synthetic locations
// Note: 'type' field not available on synthetic_location entity
// Use entity.name patterns to identify private vs public locations
fetch dt.entity.synthetic_location
| summarize {count = count()}
| fields count
```

5. Configuration

Creating a Private Location

1. **Deploy ActiveGate** with synthetic capability
2. **Navigate to:** Settings → Synthetic → Private synthetic locations
3. **Create location:** Name, description, geographic info
4. **Assign ActiveGates:** Select which ActiveGates serve this location

Location Settings

Setting	Description
Name	Descriptive location name
Latitude/Longitude	Geographic coordinates
City/Region	Location metadata
ActiveGate nodes	Assigned ActiveGates

High Availability

For production workloads:

- Deploy 2+ ActiveGates per location
- Distribute across availability zones
- Load balancing automatic

```
// Monitor executions by location
// Private locations typically have custom names (not city names like "N.
Virginia")
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| summarize {
    executions = count(),
    availability_pct = round(countIf(result.state == "SUCCESS") * 100.0 /
count(), decimals: 2)
    }, by: {monitor.name, dt.entity.synthetic_location}
| fieldsAdd location = entityName(dt.entity.synthetic_location)
| sort executions desc
| limit 30
```

6. Monitoring Private Location Health

ActiveGate Health Indicators

Metric	Description	Alert Threshold
Connectivity	Connection to cluster	Any disconnect
CPU Usage	Processing load	> 80% sustained
Memory	RAM utilization	> 80%
Disk	Storage usage	> 80%
Queue Depth	Pending executions	> 100

```
// Synthetic location health status (1 = healthy)
// Surfaces location/ActiveGate availability independent of monitor results
timeseries health = avg(dt.synthetic.location.health_status),
  from: now() - 24h, interval: 1h, by: {dt.entity.synthetic_location}
```

```
// Location availability over time (last 7 days)
fetch dt.synthetic.events, from: now() - 7d
| filter endsWith(event.type, "_monitor_execution")
| makeTimeseries {
  success_count = countIf(result.state == "SUCCESS"),
  total_count = count()
}, interval: 1h, by: {dt.entity.synthetic_location}
| fieldsAdd availability_pct = success_count[] * 100.0 / total_count[]
```

```
// Execution count by location
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| summarize {
  total_executions = count(),
  failed_executions = countIf(result.state == "FAIL"),
  unique_monitors = countDistinct(dt.synthetic.monitor.id)
}, by: {dt.entity.synthetic_location}
| fieldsAdd failure_rate = round((failed_executions * 100.0) /
total_executions, decimals: 2)
| fieldsAdd location = entityName(dt.entity.synthetic_location)
| sort total_executions desc
```


7. Troubleshooting

Common Issues

Issue	Symptoms	Resolution
No connectivity	Location offline	Check ActiveGate logs, network
SSL errors	Certificate failures	Install CA certs on ActiveGate
Timeouts	All tests fail	Check network routes, DNS
Resource exhaustion	Slow/failed tests	Scale ActiveGate resources
Browser issues	Clickpath failures	Check Chrome/display config

ActiveGate Logs

```
# Linux log location
/var/log/dynatrace/gateway/

# Key log files
gateway.log      # Main gateway log
synthetic.log    # Synthetic execution log
connection.log   # Cluster connectivity
```

Network Verification

```
# Test connectivity to target
curl -v https://internal-api.company.com/health

# DNS resolution
nslookup internal-api.company.com

# Test Dynatrace connectivity
curl -v https://<tenant>.live.dynatrace.com/api/v1/time
```

```
// Identify failing monitors by location
// To filter for specific private locations, add: | filter
dt.entity.synthetic_location == "SYNTHETIC_LOCATION-..."
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "FAIL"
| summarize {
    failure_count = count(),
    last_failure = max(timestamp)
}, by: {monitor.name, dt.entity.synthetic_location}
| fieldsAdd location = entityName(dt.entity.synthetic_location)
| sort failure_count desc
| limit 20
```

Summary

In this notebook, you learned:

- ✓ **Why private locations** - Internal apps, security, compliance
 - ✓ **Architecture** - ActiveGate with synthetic engine
 - ✓ **Deployment options** - Installer, container, Kubernetes (+ `METRIC_3RD_GEN_ENABLED` for Latest tenants)
 - ✓ **Configuration** - Creating and managing locations
 - ✓ **Health monitoring** - `dt.synthetic.location.health_status` metric + execution results by location
 - ✓ **Troubleshooting** - Common issues and resolution
-

Next Steps

Continue to **SYNTH-05: Network Monitoring** to learn about synthetic network availability monitoring.

References

- [Private synthetic locations \(DT docs\)](#)
 - [Synthetic architecture and communication \(DT docs\)](#)
 - [ActiveGate \(DT docs\)](#)
 - [ActiveGate capabilities on Kubernetes \(DT docs\)](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.